

SISTEMA DE GESTIÓN ESCOLAR

Documentación Técnica del Frontend

Versión 1.0 | Junio 2026

Stack: HTML5 · CSS3 · JavaScript ES2022 · XAMPP

1. Introducción y Descripción General

El frontend del Sistema de Gestión Escolar es una aplicación web multirol desarrollada con tecnologías estándar (HTML5, CSS3 y JavaScript vanilla). No utiliza ningún framework frontend; toda la lógica de renderizado e interacción está implementada directamente en JavaScript ES2022.

La aplicación se comunica con un backend PHP/MariaDB a través de una API REST propia, usando el patrón Bearer Token para autenticación. Está diseñada para correr localmente bajo XAMPP con la URL base:

```
http://localhost/gestion_escolar/api
```

1.1 Objetivos del Frontend

- Proveer interfaces diferenciadas para tres roles: Administrador, Profesor y Estudiante.
- Gestionar el ciclo completo de sesión: login, cambio de contraseña inicial, configuración de preguntas de seguridad y logout.
- Ofrecer operaciones CRUD sobre estudiantes, profesores, materias y matrículas (rol admin).
- Permitir registro, edición y eliminación de notas por trimestre y tipo de evaluación (rol profesor).
- Mostrar al estudiante su reporte académico agrupado por trimestre, con cálculo de nota trimestral y promedio final.
- Facilitar la comunicación estudiante ↔ sistema a través de comentarios por materia.
- Incluir un centro de ayuda interactivo con búsqueda y acordeón, adaptado al rol activo.

1.2 Tecnologías Utilizadas

Tecnología	Versión / Fuente	Uso
HTML5	Estándar	Estructura de todas las páginas
CSS3	Estándar	Estilos con variables CSS, Flexbox/Grid
JavaScript	ES2022 (vanilla)	Lógica de negocio, renderizado dinámico, llamadas API
SweetAlert2	v11 (CDN)	Alertas, confirmaciones y notificaciones toast
Font Awesome	v6.0.0 (CDN)	Iconografía en toda la interfaz
Google Fonts	CDN	Fuentes: Cinzel, EB Garamond, Cormorant Garamond, Quicksand
Fetch API	Nativa	Comunicación HTTP con el backend REST

2. Estructura de Archivos

Todo el frontend reside en la carpeta frontend/ dentro del proyecto XAMPP. La estructura es la siguiente:

```
frontend/
├── index.html           # Pantalla de login y recuperación de contraseña
├── admin.html           # Panel del Administrador
├── profesor.html        # Panel del Profesor
├── estudiante.html      # Panel del Estudiante
├── cambiar_password.html # Flujo obligatorio de cambio de contraseña
├── inicial
├── configurar_preguntas.html # Configuración de preguntas de seguridad
├── ayuda.html           # Centro de ayuda interactivo
├── js/
│   ├── api.js           # Helper HTTP compartido (fetch + token)
│   ├── login.js         # Lógica de login y recuperación de contraseña
│   ├── admin.js         # Lógica completa del panel admin
│   ├── profesor.js      # Lógica completa del panel profesor
│   ├── estudiante.js    # Lógica completa del panel estudiante
│   ├── cambiar_password.js # Lógica del formulario de cambio de contraseña
│   └── configurar_preguntas.js # Lógica del formulario de preguntas de
seguridad
├── └── ayuda.js         # Lógica del centro de ayuda (acordeón + búsqueda)
└── css/
    ├── login.css        # Estilos para login, recuperación, cambio de
contraseña
    ├── admin.css        # Estilos del panel administrador
    ├── profesor.css     # Estilos del panel profesor
    ├── estudiante.css   # Estilos del panel estudiante
    └── ayuda.css        # Estilos del centro de ayuda
```

Cada archivo HTML carga primero api.js (helper compartido) y luego el JS específico del módulo. Los archivos CSS son independientes por módulo para facilitar el mantenimiento.

3. Helper de API (api.js)

El archivo js/api.js es el único punto de comunicación con el backend REST. Todos los demás módulos lo utilizan directamente y nunca hacen llamadas fetch directas (salvo el login inicial, que no requiere token).

3.1 Constante API_URL

Define la URL base de la API:

```
const API_URL = "http://localhost/gestion_escolar/api";
```

Esta constante debe actualizarse si el proyecto se despliega en un servidor diferente al localhost.

3.2 Función getToken()

Lee el token Bearer almacenado en localStorage bajo la clave currentUser y lo devuelve. Retorna null si no hay sesión activa. Esta función es invocada internamente por apiFetch() en cada petición.

3.3 Función apiFetch(endpoint, options)

Es el wrapper principal de fetch. Inyecta automáticamente el encabezado Authorization: Bearer <token> y el encabezado Content-Type: application/json. Comportamientos especiales:

- Si el servidor responde 401 (token inválido o expirado): elimina la sesión de localStorage y redirige automáticamente a index.html.
- Si el servidor responde 403 (sin permisos): lanza un Error con el mensaje recibido del servidor.
- Devuelve la respuesta fetch original para que el caller decida cómo procesarla.

3.4 Función apiGet(endpoint)

Atajo para peticiones GET que esperan una respuesta exitosa. Lanza error si la respuesta no es ok. Devuelve directamente el campo data del JSON (o el JSON completo si no existe data), simplificando la lectura de listas y objetos.

3.5 Sesión en localStorage

Al iniciar sesión, se almacena el siguiente objeto bajo la clave currentUser:

```
{ email, token, rol, nombre, id_referencia, password_cambiada, preguntas_configuradas }
```

Este objeto es leído por todos los módulos para verificar el rol, obtener el token y controlar los flujos de onboarding.

4. Flujo de Autenticación y Onboarding

El sistema tiene un flujo de onboarding de tres pasos para usuarios nuevos (profesores y estudiantes). El administrador está exento de este flujo y accede directamente a su panel.

4.1 Pantalla de Login ([index.html](#) / [login.js](#))

Permite el acceso mediante email y contraseña. Al enviar el formulario:

- Llama a POST [/auth/login.php](#) con las credenciales.
- Si las credenciales son correctas, almacena los datos de sesión en `localStorage`.
- Redirige al administrador directamente a [admin.html](#).
- Para profesor/estudiante, evalúa los flags `password_cambiada` y `preguntas_configuradas` para determinar si debe pasar por el flujo de onboarding o acceder directamente al panel.

4.2 Cambio de Contraseña Inicial ([cambiar_password.html](#) / [cambiar_password.js](#))

Se activa cuando `password_cambiada` es false. Solicita la contraseña actual (la inicial asignada por el admin) y la nueva contraseña (mínimo 6 caracteres). Incluye el botón 'Más tarde' que establece el flag `password_skipped` en `localStorage` para permitir el acceso sin haber cambiado la contraseña, y redirige al siguiente paso del onboarding.

4.3 Configuración de Preguntas de Seguridad ([configurar_preguntas.html](#) / [configurar_preguntas.js](#))

Se activa cuando `preguntas_configuradas` es false. Muestra un grid de 3 columnas con 3 selectores de pregunta + campo de respuesta. Las preguntas se cargan desde GET [/auth/preguntas_seguridad.php](#). Valida que las 3 preguntas sean distintas y que todas tengan respuesta antes de enviar el formulario a POST [/auth/configurar_preguntas.php](#).

4.4 Recuperación de Contraseña ([index.html](#) / [login.js](#))

Accesible desde el enlace '¿Olvidaste tu contraseña?' en la pantalla de login. Flujo en dos pasos:

- Paso 1: El usuario ingresa su número de identificación. El sistema consulta GET [/auth/preguntas_usuario.php](#) y selecciona una pregunta de seguridad aleatoria.
- Paso 2: El usuario responde la pregunta y define su nueva contraseña. Se envía todo a POST [/auth/recuperar_password.php](#).

4.5 Cierre de Sesión

Disponible en todos los paneles mediante el botón 'Cerrar sesión'. Llama a POST `/auth/logout.php` para invalidar el token en el servidor, luego elimina `currentUser` de `localStorage` y redirige a `index.html`.

5. Panel del Administrador (admin.html / admin.js)

El panel del administrador es el más completo del sistema. Gestiona las cuatro entidades principales: Estudiantes, Profesores, Materias y Matrículas. Todas las tablas soportan paginación server-side y búsqueda en tiempo real.

5.1 Dashboard

Vista inicial que muestra cuatro tarjetas de estadísticas con el total de estudiantes, profesores, materias y matrículas registradas en el sistema. También incluye una sección de 'Actividad Reciente' que registra las últimas 15 acciones realizadas durante la sesión actual (agregar, editar, eliminar registros).

5.2 Gestión de Estudiantes

La tabla de estudiantes muestra: número de fila, identificación, contraseña inicial (como badge), nombre, email, grado y sección. Cada fila tiene tres acciones:

- Editar (ícono lápiz): abre el modal con los datos del estudiante precargados para modificación.
- Restablecer acceso (ícono llave): genera una nueva contraseña aleatoria y actualiza el hash en la base de datos. El usuario deberá repetir el flujo de onboarding completo en su próximo ingreso.
- Eliminar (ícono papelera): solicita confirmación y elimina el registro. No procede si el estudiante tiene notas registradas.

El modal de creación/edición incluye: número de identificación, contraseña inicial (con botón de regenerar aleatoriamente y botón de edición manual), nombre, email, grado (selector) y sección.

5.3 Gestión de Profesores

Idéntica estructura a la gestión de estudiantes. Adicionalmente muestra la columna 'Materias' con los nombres de las materias asignadas al profesor. El modal incluye el campo Especialidad y un selector múltiple de materias (solo informativo; la asignación real se hace en el formulario de materias).

5.4 Gestión de Materias

La tabla muestra: ID, código, nombre, créditos y profesor asignado. Las acciones son Editar y Eliminar. No se puede eliminar una materia que tenga calificaciones registradas. El modal permite: código único, nombre, número de créditos y selector de profesor.

5.5 Gestión de Matrículas

Tabla con búsqueda y paginación que muestra las matrículas activas. Columnas: número, nombre del estudiante + grado, nombre de la materia, nombre del profesor y fecha de asignación. La única acción disponible es Eliminar (con confirmación). El modal de creación tiene dos selectores: Estudiante y Materia.

5.6 Paginación y Búsqueda Server-Side

Todas las vistas de tabla implementan paginación y búsqueda en el servidor (no en memoria). Características:

- El campo de búsqueda tiene un listener de tipo input que reinicia la página a 1 y reconsulta la API.
- El selector 'por página' (10 / 25 / 50) afecta a la vista activa y reinicia la página.
- Los botones 'Anterior' y 'Siguiente' actualizan la variable currentXxxPage y recargan los datos.
- La función renderServerPagination() genera el HTML de paginación de forma genérica para las cuatro entidades.

5.7 Cambio de Contraseña (Admin)

Disponible en el sidebar mediante el botón 'Cambiar contraseña'. Abre un modal con tres campos: contraseña actual, nueva contraseña y confirmación. Llama a POST /auth/cambiar_password.php.

6. Panel del Profesor (profesor.html / profesor.js)

El panel del profesor permite gestionar las notas de los estudiantes matriculados en sus materias. También puede ver los comentarios que los estudiantes han dejado en sus materias.

6.1 Carga de Datos Inicial

Al cargar el panel, se realizan 4 peticiones en paralelo (Promise.all) para obtener: materias disponibles, matrículas del profesor, notas registradas y comentarios de sus materias. Los datos se filtran localmente para mostrar solo los correspondientes al profesor autenticado (usando id_referencia del token).

6.2 Dashboard del Profesor

Muestra tres tarjetas: total de materias asignadas, total de estudiantes a cargo (únicos) y total de notas registradas. Incluye una tarjeta de bienvenida con el nombre del profesor.

6.3 Mis Materias

Muestra una grilla de tarjetas, una por materia asignada. Cada tarjeta exhibe el nombre, código, créditos y número de estudiantes matriculados. Al hacer clic en una tarjeta, navega automáticamente a la vista 'Gestionar Notas' con esa materia preseleccionada.

6.4 Gestión de Notas

Vista principal del profesor. Flujo de uso:

- Seleccionar una materia en el desplegable superior.
- Opcionalmente buscar estudiantes por nombre, email o sección.
- Los estudiantes aparecen agrupados por grado en secciones colapsables.
- Cada tarjeta de estudiante muestra su promedio simple actual y estado (Aprobado / En proceso).
- Al hacer clic en un estudiante, se abre el modal de detalle de notas.

El modal de detalle muestra pestañas para I Trimestre, II Trimestre, III Trimestre y Todas. En cada pestaña:

- Se muestra la nota trimestral calculada (si están los tres componentes).
- Un formulario para agregar nueva evaluación: tipo (PARCIAL / EXAMEN_TRIMESTRAL / APRECIACION), tipo de actividad opcional (Quiz, Taller, Proyecto, etc.), nombre de actividad, nota (1.0 – 5.0) y comentario.
- El historial de notas del período con botón Eliminar por nota.
- Un resumen del trimestre con promedios de parciales, apreciación, examen y nota trimestral.

6.5 Cálculo de Notas

El sistema calcula la nota trimestral según la siguiente fórmula:

$$\text{Nota Trimestral} = (\text{Promedio Parciales} + \text{Promedio Apreciación} + \text{Examen Trimestral}) / 3$$

Solo se calcula si existen los tres componentes. El promedio final del año es el promedio de las tres notas trimestrales, y solo se calcula si los tres trimestres están completos.

6.6 Comentarios

Muestra en tarjetas todos los comentarios que los estudiantes han enviado sobre las materias del profesor. Cada tarjeta incluye: nombre del estudiante, nombre de la materia, texto del comentario y fecha de envío.

7. Panel del Estudiante (estudiante.html / estudiante.js)

El panel del estudiante es de solo lectura en cuanto a notas se refiere. Permite consultar el reporte académico y enviar comentarios a los profesores.

7.1 Dashboard del Estudiante

Muestra tres tarjetas: número de materias matriculadas, promedio general (calculado con promedio simple de todas las notas registradas) y número de materias aprobadas (promedio $\geq$ 3.0). Incluye tarjeta de bienvenida y tarjeta motivacional.

7.2 Reporte de Notas por Trimestre

Vista principal del estudiante. Organiza todas las notas en tres secciones colapsables (una por trimestre). Cada sección muestra:

- El promedio del trimestre (si todos los componentes están disponibles).
- Una tabla con las materias matriculadas, mostrando el botón 'Ver notas' (con conteo), la nota trimestral y el estado.

Al pulsar 'Ver notas', se abre un modal de detalle con:

- Encabezado con nombre de materia, trimestre y badge de nota trimestral / estado.
- Grupos de notas por tipo: Parciales (con promedio), Evaluación (con promedio) y Examen Trimestral.
- Tabla resumen con los tres promedios y la nota trimestral final.

7.3 Comentarios

Permite al estudiante enviar un comentario sobre una materia en la que esté matriculado. El formulario tiene un selector de materia y un textarea. Al enviar, la lista de comentarios propios se recarga automáticamente desde la API.

8. Centro de Ayuda (ayuda.html / ayuda.js)

Página independiente con un acordeón de preguntas frecuentes. El contenido se adapta dinámicamente al rol del usuario activo.

8.1 Secciones de Contenido

Sección	Visible para
Acceso al sistema	Todos los roles
Panel del administrador	Admin (o sin sesión)
Panel del profesor	Profesor (o sin sesión)
Panel del estudiante	Estudiante (o sin sesión)
Preguntas frecuentes	Todos (algunas filtradas por rol)

8.2 Búsqueda

El campo de búsqueda filtra en tiempo real tanto las preguntas como las respuestas (incluyendo el HTML interno de las respuestas). Las secciones vacías desaparecen del resultado.

8.3 Acordeón

Cada ítem se expande/colapsa con animación CSS usando max-height transitionado. El ícono de flecha rota 180° al expandir. Solo un ítem puede estar activo a la vez dentro de su sección.

9. Sistema de Diseño y Estilos

El sistema utiliza un tema visual consistente de estética clásica/académica con paleta de colores de carmesí, dorado e ivory (marfil), implementado con CSS custom properties (variables CSS).

9.1 Variables de Color (CSS Custom Properties)

Variable	Valor HEX	Uso
--crimson	#8B0000	Color principal, bordes activos, botones primarios
--crimson-deep	#5C0000	Títulos, encabezados, fondos oscuros
--crimson-light	#B22222	Hover de botones primarios
--gold	#C9A84C	Acentos, íconos activos, borde de cards dorados
--gold-light	#E2C97E	Textos sobre fondos oscuros
--gold-pale	#F5E6B8	Texto en botones primarios
--ivory	#FAF6EE	Fondo general de la aplicación
--ivory-dark	#F0E8D5	Fondos alternativos, secciones de tablas
--parchment	#E8DEC6	Bordes sutiles, separadores
--ink	#1A0A0A	Texto principal
--ink-soft	#3B1A1A	Texto secundario

9.2 Tipografía

Fuente	Uso
Cinzel	Títulos, encabezados, etiquetas de formulario, botones, texto de interfaz principal
EB Garamond	Cuerpo de texto, celdas de tabla, párrafos generales
Cormorant Garamond	Subtítulos en italic, información secundaria, taglines
Courier New	Contraseñas iniciales (monoespaciado para facilitar lectura)
Quicksand	Fuente alternativa en login.css (legado)

9.3 Layout

Todos los paneles principales (admin, profesor, estudiante) usan el mismo layout base:

- display: flex en .app-container para separar sidebar y contenido principal.
- Sidebar fijo (position: fixed) de ancho 260-280px con fondo oscuro degradado.
- Contenido principal con margin-left igual al ancho del sidebar y padding generoso.

- Diseño responsive que colapsa el sidebar a 68-72px en pantallas menores a 768px, ocultando textos y mostrando solo íconos.

9.4 Componentes Visuales Reutilizables

Componente	Descripción
.stat-card	Tarjeta de estadística con ícono, número grande y etiqueta. Borde superior crimson.
.data-table	Tabla con encabezado degradado crimson, filas alternadas y hover sobre fondo parchment.
.modal	Overlay semitransparente con blur, card central con bordes dorados y animación fadeIn.
.btn-primary	Botón con degradado crimson, borde dorado y hover con elevación.
.btn-edit / .btn-danger	Botones de acción de tabla (editar en dorado, eliminar en crimson).
.nav-btn	Botón de navegación del sidebar con borde izquierdo activo dorado.
.badge	Pastilla de texto para info secundaria (materias, contraseñas, grados).
.search-bar	Barra de búsqueda con borde inferior crimson y focus en dorado.
.pagination	Controles de paginación con botones anterior/siguiente y selector por página.

10. Variables y Estado Global por Módulo

Cada módulo JS mantiene su propio estado en variables globales de módulo (no hay store centralizado). A continuación se detallan las variables principales de cada archivo.

10.1 admin.js

Variable	Tipo	Descripción
students	Array	Página actual de estudiantes cargada desde la API
teachers	Array	Página actual de profesores
subjects	Array	Página actual de materias
enrollments	Array	Página actual de matrículas
activities	Array	Últimas 15 actividades de la sesión
currentStudentPage / currentProfessorPage / ...	Number	Página actual de cada entidad
perPage	Number	Registros por página (10, 25 o 50)
studentSearch / professorSearch / ...	String	Texto de búsqueda actual por entidad
studentTotal / professorTotal / ...	Number	Total de registros en servidor para cada entidad
currentActiveView	String	Vista actualmente activa (dashboard, students, etc.)

10.2 profesor.js

Variable	Tipo	Descripción
currentTeacher	Object	Usuario autenticado (leído de localStorage)
mySubjects	Array	Materias asignadas al profesor autenticado
myEnrollments	Array	Matrículas de los estudiantes del profesor
myGrades	Array	Todas las notas registradas por el profesor
myComments	Array	Comentarios de estudiantes en materias del profesor
myStudents	Array	Mapa único de estudiantes derivado de matrículas
currentSubjectId	Number null	Materia seleccionada actualmente
currentStudentForModal	Object null	Estudiante cuyo modal de notas está abierto
currentTrimestre	String	Pestaña de trimestre activa en el modal

Variable	Tipo	Descripción
studentGradeSearch	String	Texto del buscador de estudiantes en la vista de notas

10.3 estudiante.js

Variable	Tipo	Descripción
currentStudent	Object	Usuario autenticado (leído de localStorage)
myEnrollments	Array	Matrículas del estudiante
myGrades	Array	Todas las notas del estudiante (normalizadas)
mySubjects	Array	Materias en las que el estudiante está matriculado
TRIMESTRES	Array	Constante: ['I Trimestre', 'II Trimestre', 'III Trimestre']

11. Endpoints de la API Consumidos por el Frontend

La siguiente tabla resume todos los endpoints REST que el frontend consume, el método HTTP, el módulo que los llama y el rol que los puede ejecutar.

Endpoint	Método	Módulo(s)	Rol(es)
/auth/login.php	POST	login.js	Todos
/auth/logout.php	POST	admin.js, profesor.js, estudiante.js	Todos
/auth/cambiar_password.php	POST	cambiar_password.js, admin.js, profesor.js, estudiante.js	Todos
/auth/preguntas_seguridad.php	GET	configurar_preguntas.js	Todos
/auth/configurar_preguntas.php	POST	configurar_preguntas.js	Todos
/auth/preguntas_usuario.php	GET	login.js (recuperación)	Público
/auth/recuperar_password.php	POST	login.js (recuperación)	Público
/estudiantes/	GET	admin.js	Admin
/estudiantes/	POST	admin.js	Admin
/estudiantes/	PUT	admin.js	Admin
/estudiantes/delete.php	DELETE	admin.js	Admin
/profesores/	GET	admin.js	Admin
/profesores/	POST	admin.js	Admin
/profesores/	PUT	admin.js	Admin
/profesores/delete.php	DELETE	admin.js	Admin
/materias/	GET	admin.js, profesor.js, estudiante.js	Todos
/materias/	POST	admin.js	Admin
/materias/	PUT	admin.js	Admin
/materias/delete.php	DELETE	admin.js	Admin
/matriculas/	GET	admin.js, profesor.js, estudiante.js	Todos
/matriculas/	POST	admin.js	Admin
/matriculas/	DELETE	admin.js	Admin
/notas/	GET	profesor.js, estudiante.js	Profesor, Estudiante
/notas/	POST	profesor.js	Profesor
/notas/	DELETE	profesor.js	Profesor

Endpoint	Método	Módulo(s)	Rol(es)
/comentarios/	GET	profesor.js, estudiante.js	Profesor, Estudiante
/comentarios/	POST	estudiante.js	Estudiante

12. Consideraciones de Seguridad en el Frontend

12.1 Escape de HTML

Todos los módulos implementan una función `escapeHtml(str)` que reemplaza los caracteres `&`, `<` y `>` por sus entidades HTML correspondientes (`&`, `<`, `>`). Esta función se aplica a todos los datos provenientes de la API antes de insertarlos en el DOM mediante `innerHTML`, previniendo ataques XSS.

12.2 Verificación de Rol

Cada módulo verifica al inicio que el usuario almacenado en `localStorage` tenga el rol correcto. Si no es así, redirige inmediatamente a `index.html`. Sin embargo, esta verificación es solo una capa de UX; la seguridad real está implementada en el backend mediante el middleware de autenticación.

12.3 Manejo de Token Expirado

La función `apiFetch()` intercepta cualquier respuesta 401 del servidor, elimina la sesión de `localStorage` y redirige al login. Esto garantiza que las sesiones expiradas sean manejadas de forma transparente para el usuario.

12.4 Contraseñas

Las contraseñas nunca se almacenan en `localStorage` ni se muestran en texto plano en el DOM (excepto la contraseña inicial del admin en las tablas, que es una contraseña temporal que el usuario debe cambiar). La comunicación con el backend se realiza en texto plano por HTTP local (XAMPP); en producción sería necesario usar HTTPS.

12.5 Caché del Navegador

Los archivos CSS y JS incluyen comentarios que documentan cómo manejar problemas de caché del navegador durante el desarrollo. Se recomienda usar parámetros de versión (`?v=N`) en los atributos `src` y `href` para forzar la recarga cuando se hacen cambios.

13. Guía de Mantenimiento y Extensión

13.1 Cambiar la URL de la API

La constante `API_URL` en `js/api.js` es el único punto a modificar para apuntar el frontend a un servidor diferente. No hay otra referencia a la URL base en el código.

13.2 Agregar un Nuevo Campo a Estudiantes o Profesores

- Agregar el campo `<input>` en el modal HTML correspondiente (`admin.html`).
- Incluir el nuevo campo en el objeto `payload` dentro del listener `submit` del formulario en `admin.js`.
- Actualizar la función `renderStudents()` o `renderTeachers()` para mostrar la nueva columna en la tabla.
- Actualizar las funciones `editStudent()` / `editTeacher()` para precargar el valor en el modal de edición.

13.3 Agregar una Nueva Vista a un Panel

- Agregar un botón `.nav-btn` con `data-view='miVista'` en el sidebar del HTML.
- Agregar el `div#miVistaView.view` con el contenido en el HTML.
- En el listener de navegación de JS, agregar el caso `if (view === 'miVista')` para cargar los datos.

13.4 Modificar el Cálculo de Notas

Las funciones de cálculo están bien aisladas tanto en `profesor.js` como en `estudiante.js`:

- `calcularNotaTrimestral()`: calcula la nota de un trimestre dado.
- `calcularPromedioFinal()`: promedia las tres notas trimestrales.
- `calcularPromedioSimple()`: promedio simple de todas las notas (para el dashboard).
- `generarResumenTrimestre()`: devuelve el objeto con todos los parciales de un trimestre.

Para cambiar la fórmula, solo hay que modificar estas funciones en ambos archivos.

13.5 Añadir Preguntas de Seguridad

Las preguntas de seguridad se gestionan exclusivamente en la base de datos (tabla `pregunta_seguridad`). El frontend las carga dinámicamente desde el endpoint `/auth/preguntas_seguridad.php`. No requiere cambios en el frontend.
